

Transformers

Da teoria à implementação

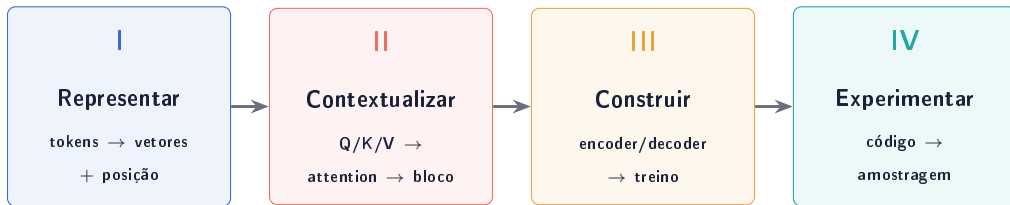
Samir Braga Chaves

2026



A rota da aula

Como o texto se transforma em previsões de um Transformer?

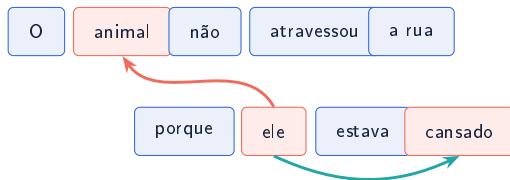


Cada etapa responde a uma pergunta e fornece a entrada da seguinte.

Por que Transformers?

Sequências carregam contexto

Para interpretar “ele”, quais palavras oferecem pistas?



Ao atualizar o vetor de **ele**, o mecanismo *self-attention* pode combinar **animal** e **cansado**, mesmo que estejam distantes.

Exemplo bidirecional, como em um encoder: tokens à esquerda e à direita podem ser consultados.

O desafio

- preservar a ordem;
- resolver referências;
- combinar pistas distantes;
- fazer isso em sequências longas.

RNNs percorrem; Transformers conectam

RNN



um passo depende
do anterior

Transformer



posições processadas
juntas na camada

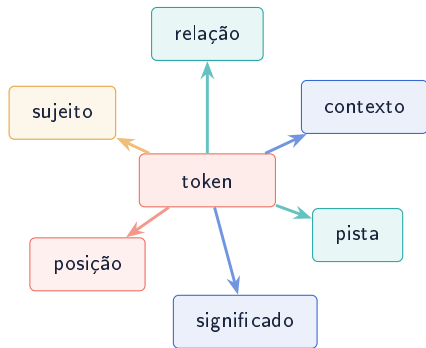
RNN/LSTM

passos sequenciais e
caminho longo entre tokens

Transformer

conexão direta na camada

A mudança de perspectiva



Attention

torna-se o mecanismo principal de
comunicação entre tokens.

Vaswani et al., 2017
Attention Is All You Need

I — Representando a entrada

Modelos recebem tokens, não frases



O tokenizador escolhe uma segmentação e consulta um vocabulário.

Token

unidade disponível no vocabulário

ID

endereço inteiro dessa unidade

Três granularidades possíveis



A escolha controla o tamanho do vocabulário, o comprimento da sequência e como palavras raras são representadas.

Subwords: três caminhos para um objetivo

BPE

e + s → es
es + t → est

funde pares frequentes

WordPiece

play + ##ing
transform + ##er

favorece segmentações prováveis

Unigram

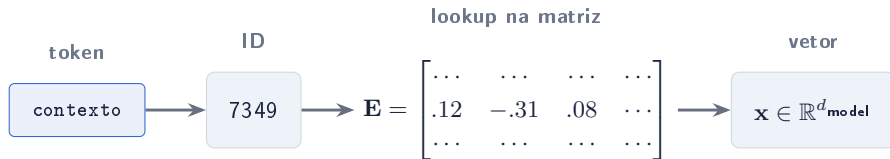
várias segmentações
↓ probabilidade

remove peças menos úteis



[Exemplo interativo: teste sua própria frase no OpenAI Tokenizer](#) ↗

Do ID a um vetor aprendido



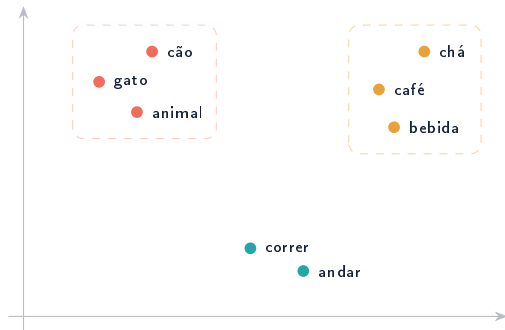
$\mathbf{E} \in \mathbb{R}^{|V| \times d_{\text{model}}}$: uma linha por token do vocabulário.

O ID é apenas o endereço; o vetor é aprendido durante o treino.

d_{model} é a largura interna das representações: cada token é descrito por d_{model} números, dimensão mantida entre os blocos.

[Documentação: camada `torch.nn.Embedding`](#) ↗

Um espaço que organiza usos semelhantes



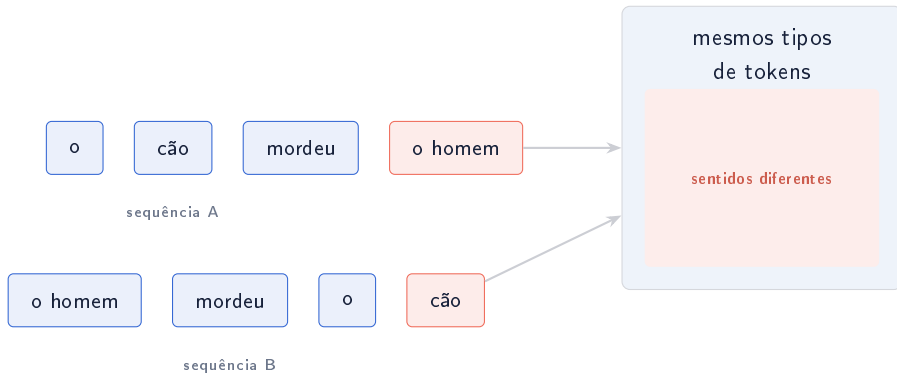
Contextos parecidos podem aproximar vetores

Tokens que aparecem em contextos parecidos podem adquirir vetores próximos ou direções relacionadas.

geometria aprendida

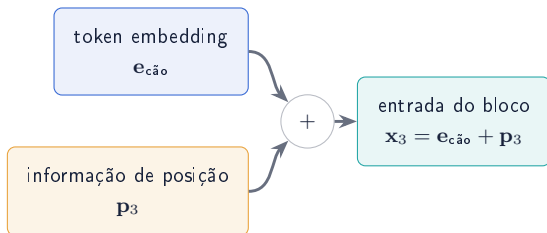
O desenho é uma projeção ilustrativa, não um mapa literal do espaço completo.

Sem posição, self-attention plena não codifica ordem



Sem posição e sem uma máscara dependente da ordem, permutar os tokens apenas permuta as saídas.
Precisamos codificar **onde** cada token está.

A posição entra junto com o conteúdo



Como representar posição?

Fixa um vetor p_i de senos e cossenos é somado.

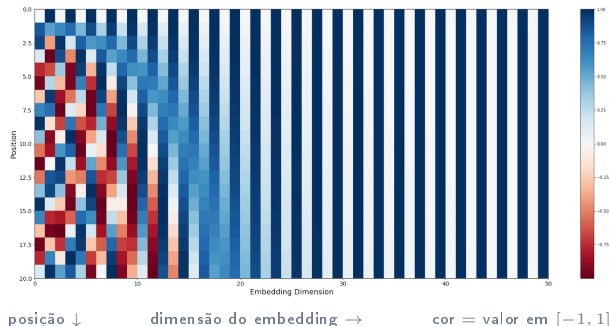
Aprendida um vetor p_i ajustável é somado.

RoPE - Rotary Position Embedding não soma p_i : aplica rotações dependentes da posição a Q e K .

No Transformer original, p_i é **fixo e senoidal**. O próximo slide explica a regra.

Ondas em várias frequências codificam cada posição

Cada linha é um vetor de posição $\mathbf{p}_{pos}$

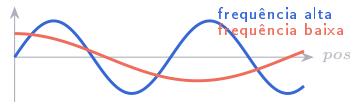


Regra do Transformer original

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

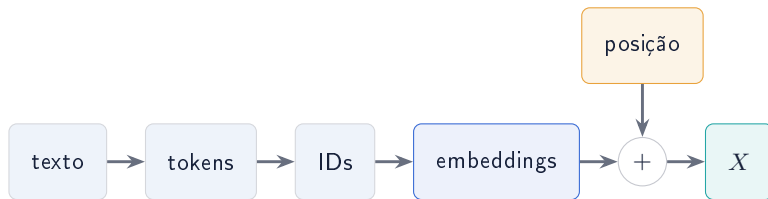
Duas frequências entre as várias dimensões



Fixando uma linha pos e percorrendo as colunas, obtemos $\mathbf{p}_{pos} \in \mathbb{R}^{d_{\text{model}}}$. Em cada par, seno e cosseno compartilham uma frequência; pares com outros valores de i usam frequências diferentes.

Fonte: Visualização adaptada de Özgen (2021).

Até aqui: construímos a entrada



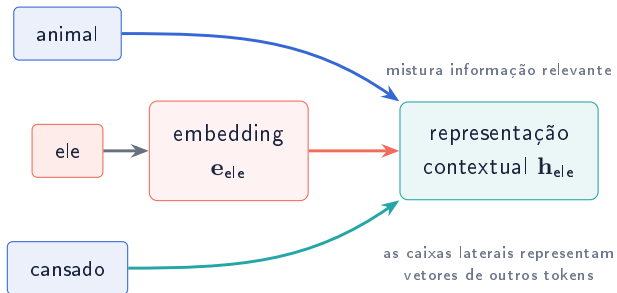
Próxima pergunta:

Como os vetores em X trocam informação para adquirir contexto?

Resposta: **attention**

II — O coração do Transformer

O contexto transforma o vetor de cada token



Antes

o vetor representa o token.

Depois

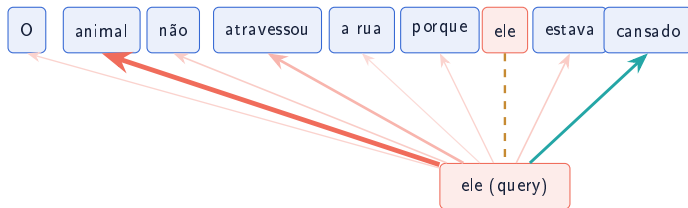
o vetor representa o token **naquela sequência**.

Pesos de attention não são, sozinhos, explicações causais.

Aqui a attention é bidirecional.

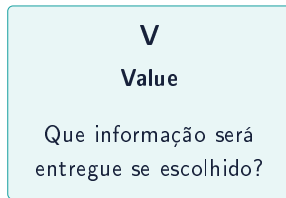
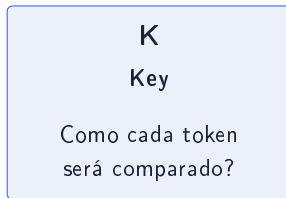
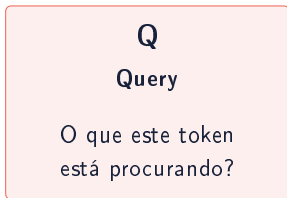
Attention é uma consulta ao contexto

Exemplo bidirecional: “ele” pode consultar tokens à esquerda e à direita.



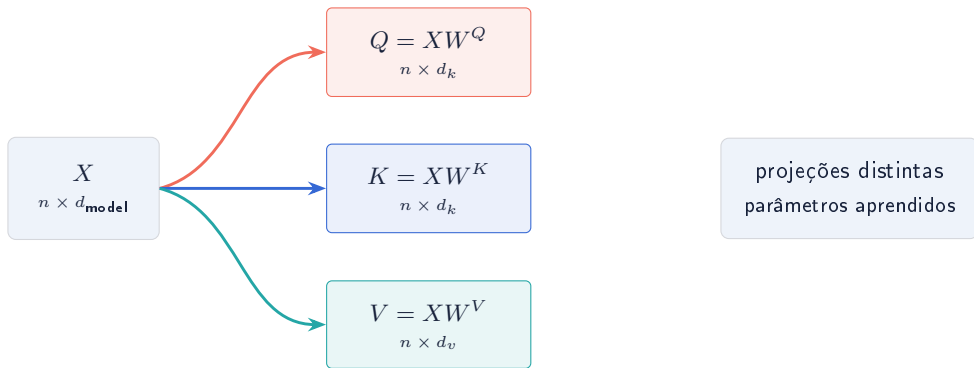
Cada token atribui pesos diferentes aos tokens que pode consultar e usa esses pesos para combinar informação.

Q, K e V têm papéis diferentes



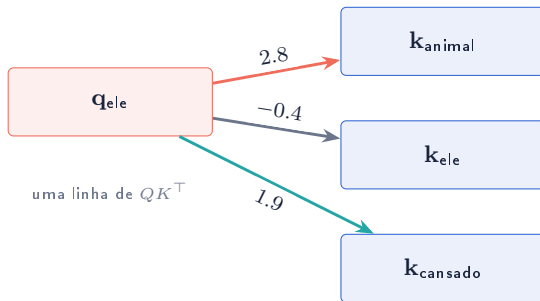
A mesma entrada gera três visões aprendidas. *Q* é comparado com *K*; os pesos resultantes determinam quanto conteúdo de *V* será misturado.

Uma attention projeta X em Q , K e V



Em self-attention, uma mesma sequência gera três projeções aprendidas. Mais adiante, cada *head* receberá seu próprio conjunto de projeções.

O produto escalar mede compatibilidade

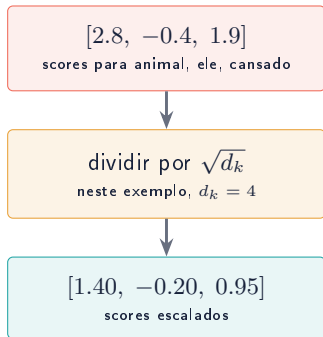


$$s_{ij} = \mathbf{q}_i \mathbf{k}_j^\top$$

Score maior significa maior **compatibilidade aprendida** entre aquela query e aquela key.

O score ainda não é probabilidade.

A escala evita que o softmax sature



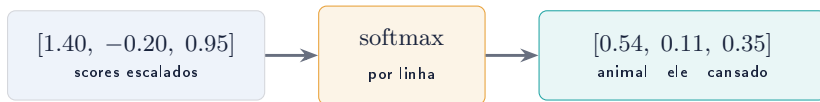
Sob a hipótese simplificada de componentes independentes, com média zero e variância unitária,

$$\text{Var}(\mathbf{q}\mathbf{k}^\top) = d_k.$$

Sem a escala, dimensões maiores produzem logits grandes, softmax muito concentrado, quase one-hot, e gradientes pequenos.

A partir daqui, seguiremos estes mesmos três números até a mistura dos Values.

Softmax transforma scores em pesos



Propriedades dos pesos após o softmax

não negativos

$$0 \leq \alpha_{ij} \leq 1$$

soma 1

para cada query, $\sum_j \alpha_{ij} = 1$

A diagonal é o peso de cada ocorrência sobre si mesma

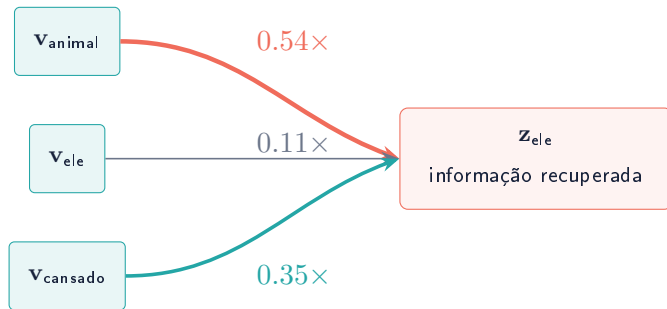
		keys		
		animal	ele	cansado
queries	animal			
	ele	0.54	0.11	0.35
	cansado			

Em self-attention, os dois eixos listam as mesmas ocorrências, na mesma ordem. A linha da query **ele** contém os pesos do slide anterior.

diagonal $\alpha_{\text{ele},\text{ele}} = 0.11$ é o peso de “ele” sobre sua própria ocorrência. Ele é aprendido e não precisa ser 0 nem 1.

$$\alpha_{ij} = \text{peso de } i \text{ sobre } j, \quad \sum_j \alpha_{ij} = 1$$

A saída é uma mistura dos Values



$$\mathbf{z}_i = \sum_j \alpha_{ij} \mathbf{v}_j$$

Scores escolhem *quanto*; Values determinam o *que* é combinado.

A máscara define quais keys cada query pode consultar



A máscara M é somada aos scores quando nem toda key deve estar disponível para toda query.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}} + M\right) V$$

$M_{ij} = 0$ mantém a comparação; $M_{ij} = -\infty$ faz seu peso tornar-se zero após o softmax.

Padding

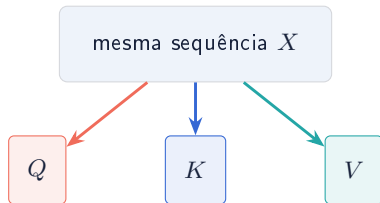
Em lotes com sequências de comprimentos diferentes, impede que tokens reais consultem posições artificiais [PAD].

Causal

No decoder autoregressivo, impede que a posição t veja tokens futuros e receba antecipadamente a resposta que deve prever.

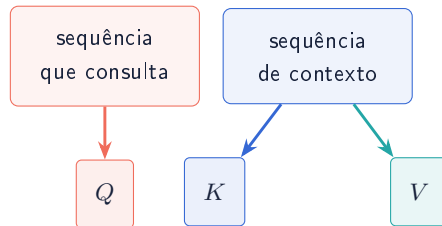
Self e cross-attention diferem pela origem

Self-attention



Q , K e V vêm da própria sequência.

Cross-attention



Q consulta informação oferecida por outra sequência.

No Transformer original, a sequência que consulta é o decoder; a sequência de contexto é a memória produzida pelo encoder.

Uma head oferece uma lente sobre a sequência

Uma *head* é uma attention completa, com projeções W_i^Q , W_i^K e W_i^V próprias.

Head 1

ele → animal

possível correferência

Head 2

atravessou → rua

possível relação local

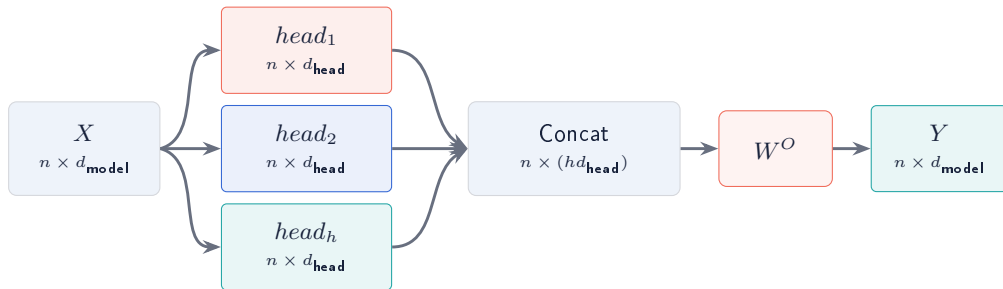
Head 3

cansado → ele

possível relação prediativa

Diferentes heads usam projeções próprias e **podem** especializar-se em relações complementares — isso não é garantido nem necessariamente interpretável.

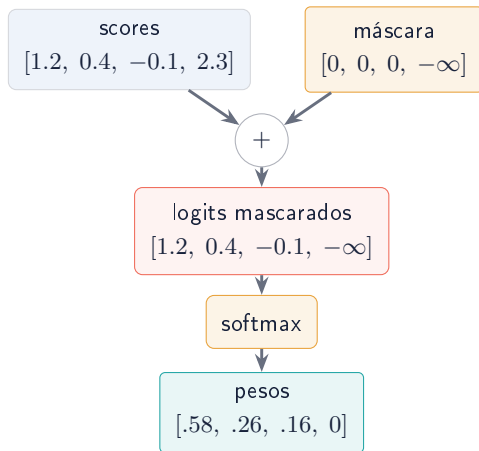
Heads projetam subespaços; a concatenação recompõe a largura



$$head_i = \text{Attention}(Q_i, K_i, V_i), \quad \text{MultiHead} = \text{Concat}(head_1, \dots, head_h) W^O$$

Se $d_{\text{head}} = d_{\text{model}}/h$, então $h d_{\text{head}} = d_{\text{model}}$. No Transformer original: $512 \rightarrow 8 \times 64 \rightarrow 512$ — uma configuração, não uma regra universal.

Máscaras alteram logits antes do softmax



Padding mask

lotes usam [PAD] para igualar comprimentos; a máscara impede consultar essas posições artificiais.

Causal mask

impede consulta a posições futuras.

As duas máscaras podem ser combinadas.

A causal mask bloqueia o futuro

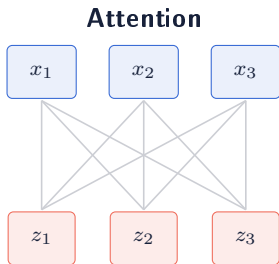
	k_1	k_2	k_3	k_4	k_5
q_1	0	$-\infty$	$-\infty$	$-\infty$	$-\infty$
q_2	0	0	$-\infty$	$-\infty$	$-\infty$
q_3	0	0	0	$-\infty$	$-\infty$
q_4	0	0	0	0	$-\infty$
q_5	0	0	0	0	0
queries					
					keys

Na linha t , apenas colunas $j \leq t$ permanecem visíveis.

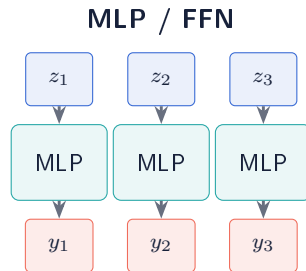
$$M_{tj} = \begin{cases} 0, & j \leq t, \\ -\infty, & j > t. \end{cases}$$

O softmax transforma posições bloqueadas em peso zero.

Attention comunica entre posições; o MLP transforma cada posição



informação cruza posições

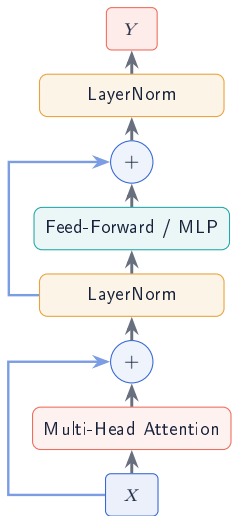


mesma MLP aplicada a cada posição

$$\text{FFN}(x) = \sigma(xW_1 + b_1)W_2 + b_2$$

Features são as coordenadas do vetor; σ é uma não linearidade, como GELU. Os mesmos pesos são reutilizados em todas as posições.

Residuais contornam cada subcamada



No Transformer de 2017:

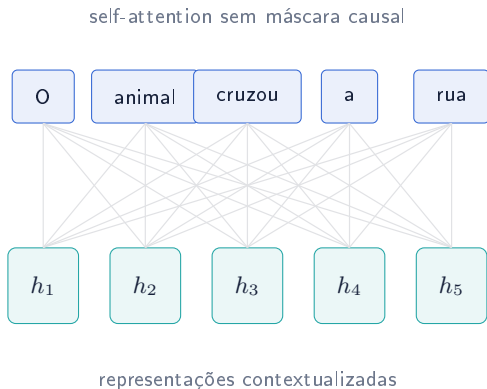
$$\text{LN}(x + \text{Sublayer}(x))$$

- attention comunica entre posições;
- MLP transforma cada posição;
- residual preserva o sinal;
- LayerNorm estabiliza o fluxo.

Muitos modelos modernos usam a ordem pre-norm.

III — Construindo e treinando modelos

O encoder contextualiza toda a entrada

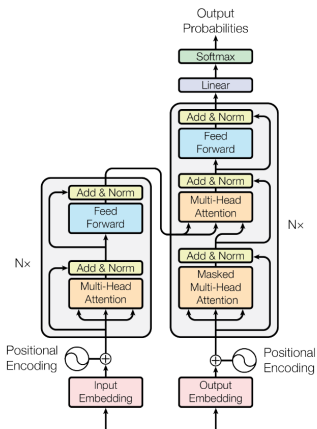


Cada posição pode consultar tokens à **esquerda e à direita**.

- contexto global por camada;
- saída para cada token;
- útil para compreender a sequência.

“Bidirecional” descreve o padrão de atenção; não significa duas RNNs.

O Transformer original conecta encoder e decoder



Como ler a figura

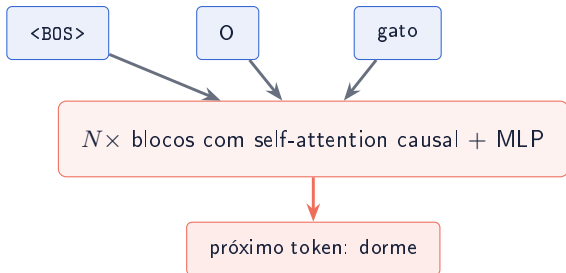
Encoder coluna esquerda: self-attention plena e feed-forward.

Decoder coluna direita: self-attention mascarada, attention sobre o encoder e feed-forward.

Cross-attention é o segundo bloco Multi-Head Attention do decoder. Suas queries vêm do decoder; keys e values vêm da saída do encoder.

O artigo original usa o nome *encoder-decoder attention*.

Sem encoder e cross-attention, resta o decoder-only



O decoder consulta somente o prefixo disponível:

$$p(x_{t+1} \mid x_{\leq t}).$$

Na geração, o token escolhido volta à entrada; a sequência cresce **um token por vez**.

Esta simplificação do diagrama anterior é a forma usada por muitos LLMs generativos e pelo código do hands-on.

A cabeça de saída devolve o vetor ao vocabulário



Durante o treino

A cross-entropy compara o token-alvo com os logits. Seu gradiente ajusta a cabeça de saída e, por retropropagação, embeddings, $W^Q/W^K/W^V$, MLPs e demais parâmetros treináveis.

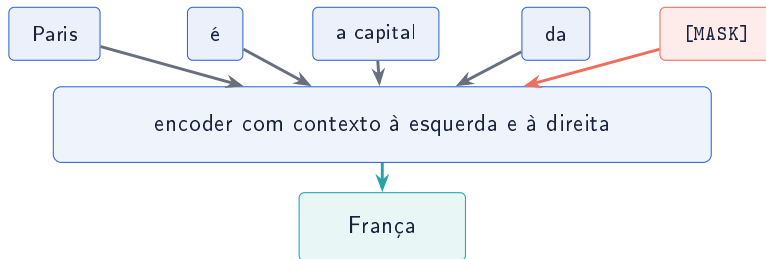
Durante a geração

Greedy usa $\arg \max$ dos logits. Amostragem converte logits em probabilidades com softmax e sorteia. O token escolhido volta ao contexto.

$|V|$ logits conectam a dimensão interna d_{model} ao vocabulário. <BOS> marca o início; <EOS> pode encerrar a geração, que também pode parar por limite de comprimento.

Fonte: Vaswani et al. (2017); sequência visual inspirada em Alammar (2018).

BERT aprende reconstruindo tokens ocultos

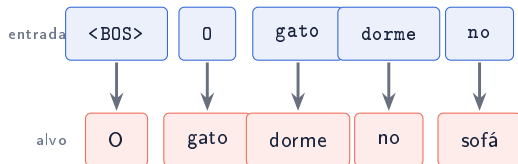


contexto corrompido $\rightarrow p(\text{token original} \mid \text{contexto})$

A loss de MLM é calculada nas posições selecionadas para predição. BERT original também usa Next Sentence Prediction.

Fonte: Devlin et al. (2019).

GPT aprende a prever o próximo token



$$p(x_{1:T}) = \prod_{t=1}^T p(x_t | x_{<t})$$

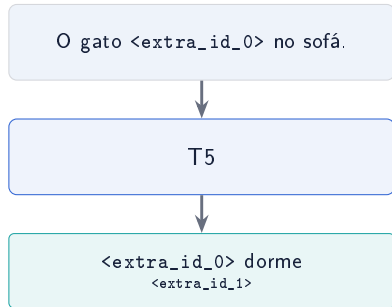
A mesma loss é calculada para todas as posições conhecidas da sequência.

decoder-only

Fonte: Radford et al. (2018).

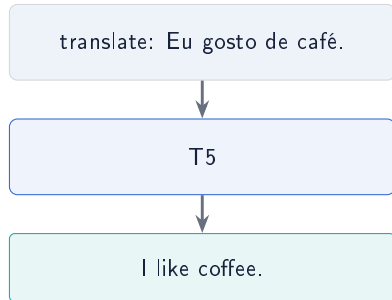
T5 formula tarefas de linguagem como texto para texto

Pré-treinamento



span corruption: cada <extra_id_k> marca um trecho removido; o alvo lista esses trechos na ordem

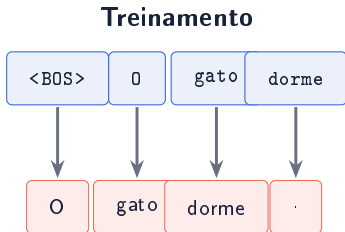
Tarefa downstream



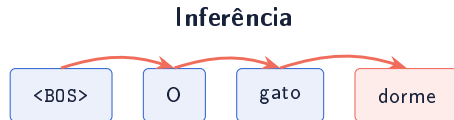
tradução, resumo, classificação, QA...

Fonte: Raffel et al. (2020).

Treino paralelo; geração sequencial



uma passagem; quatro alvos



uma passagem por novo token

No treino, todos os alvos são conhecidos e entram deslocados; a causal mask impede vazamento do futuro, mas todas as posições da camada são computadas juntas.

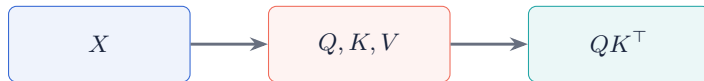
O Transformer encurta o caminho sequencial

RNN



$O(n)$ passos sequenciais

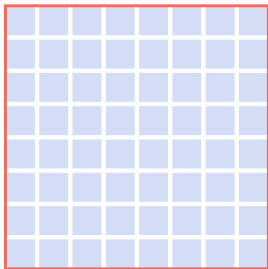
Transformer



operações matriciais para todas as posições

GPUs e TPUs calculam muitas posições simultaneamente **dentro de cada camada**. As camadas continuam encadeadas e a geração autoregressiva continua sequencial.

Por camada, attention cresce quadraticamente com a sequência



matriz de scores $n \times n$

Operação por camada	Custo principal
scores de attention	$O(n^2 d)$
memória dos scores	$O(hn^2)$
projeções + MLP	$O(nd^2)$

n : comprimento; d : dimensão; h : número de heads.

Para a MLP, $d_{ff} = \Theta(d)$.

Dobrar o comprimento multiplica por quatro o número de pares token-token. Métodos modernos reduzem esse custo, mas a attention convencional é quadrática em n .

Fonte: Complexidades conforme Vaswani et al. (2017), Seção 4.

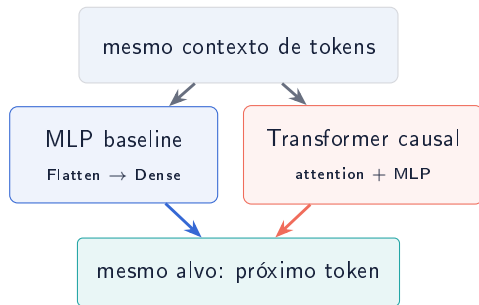
Três famílias organizam atenção e fluxo

Arquitetura	Padrão principal	Exemplo	Uso típico
Encoder-only	self-attention sem máscara causal	BERT	compreensão de texto
Decoder-only	self-attention causal	GPT	geração de texto
Encoder-decoder	encoder pleno; decoder causal + cross-attention	T5	transformação seq2seq

Muitos dos LLMs generativos mais conhecidos são decoder-only, mas as três famílias continuam relevantes. Os usos são típicos, não exclusivos.

IV — Hands-on: do slide ao código

O repositório isola a diferença entre duas arquiteturas



mesma tokenização, loss, avaliação e amostragem

Objetivo didático

comparar **como o contexto é combinado**, mantendo o restante do experimento constante.

- Tiny Shakespeare por caractere;
- TinyStories com BPE opcional;
- JAX, Flax NNX e Optax como stack;
- experimento didático, não estado da arte.

Para acompanhar: [acesso ao repositório](#) e, para gerar texto, um [checkpoint treinado](#).

A implementação é um decoder-only compacto



$T = 64$

janela de contexto

$D = 128$

dimensão do modelo

$h = 4, L = 2$

heads e blocos

$d_{ff} = 512$

largura da MLP

B : exemplos no lote; T : tokens por janela; D : dimensão interna; V : tamanho do vocabulário. O código usa apenas $x[:, -1, :]$ para prever um alvo por janela; o próximo slide explica essa escolha deliberada.

Cada bloco usa pre-LN, attention causal, residual, Linear-GELU-Linear e dropout. encode também retorna mapas $[B, h, T, T]$; as posições são aprendidas e não há encoder nem cross-attention.

Fonte: `transformer.py`.

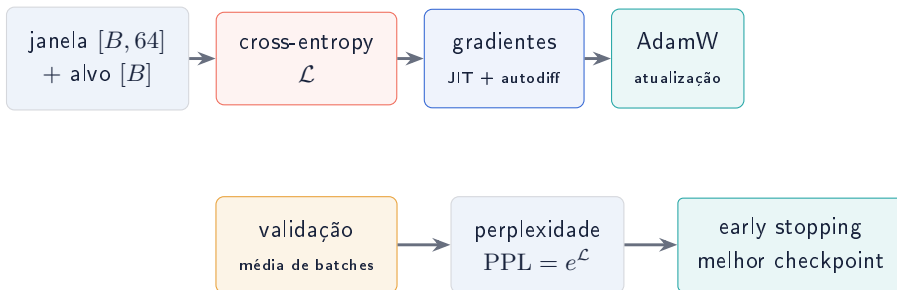
Quatro escolhas explicam por que slides e código não são idênticos

Nos slides	Na implementação	Por quê?
bloco original post-norm	bloco pre-norm	variante comum em modelos modernos; muda a ordem, não os componentes
Q , K e V separados	uma Linear $D \rightarrow 3D$ e reshape	projeção fundida é computacionalmente conveniente e algebricamente equivalente
loss em todas as posições	um alvo após cada janela	permite comparação direta com o MLP baseline, mas subutiliza o Transformer
encoder, decoder e cross-attention	apenas decoder causal	o experimento estuda modelagem autoregressiva, não todas as famílias

O código é uma **instância didática** da arquitetura, não uma reprodução literal de todos os modelos apresentados.

Fonte: `transformer.py` e `train.py`.

Treino e avaliação tornam a comparação reproduzível

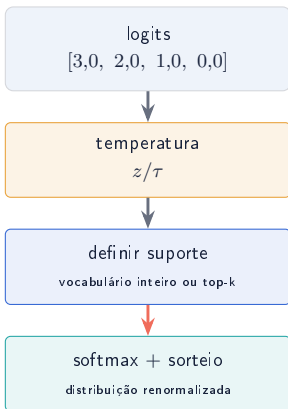


$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \log p(y_i | \mathbf{x}_i)$$

Aqui B é o número de exemplos no lote e $\mathcal{L}$ é a cross-entropy média por alvo. Dropout (0,1) atua no treino e é desligado em validação e geração; o checkpoint guarda parâmetros, não todo o otimizador.

Fonte: `train.py` e `checkpoint.py`.

Amostragem transforma logits em escolhas



Referência: greedy

escolhe sempre $\arg \max_i z_i$; é determinístico, mas não é a estratégia usada na comparação principal.

Multinomial

sorteia no vocabulário inteiro segundo p_i .

Top-k

mantém os k maiores logits e sorteia apenas entre eles.

A geração continua autoregressiva: o token sorteado volta para o contexto da próxima passagem.

Fonte: `generation.py`.

Temperatura muda a concentração, não o ranking

$$p_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}, \quad \tau > 0$$

$\tau = 0,5$

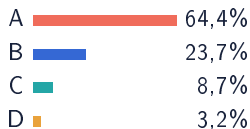
distribuição concentrada



conservadora; pode repetir

$\tau = 1$

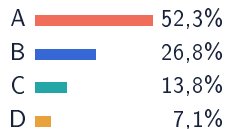
softmax original



preserva a distribuição

$\tau = 1,5$

distribuição mais plana



diversa; aumenta o risco

Mesmos logits $[3, 2, 1, 0]$: dividir por $\tau > 0$ não altera sua ordem, apenas as diferenças relativas após o softmax.

Top-k corta a cauda e renormaliza



$$\mathcal{K} = \text{TopK}(z, k), \quad p'_i = \frac{\exp(z_i/\tau)}{\sum_{j \in \mathcal{K}} \exp(z_j/\tau)} \quad (i \in \mathcal{K}), \quad p'_i = 0 \quad (i \notin \mathcal{K}).$$

No experimento: $\tau = 0,7$ e $k = 12$. τ redistribui pesos; k restringe o conjunto sorteável.

Fonte: `generation.py`.

Hands-on: mude um fator por vez

```
uv sync
```

```
uv run -m src.transformers_class.run_tiny_shakespeare -mode train
```

```
uv run -m src.transformers_class.run_tiny_shakespeare -mode generate -seed 42
```

O modo `generate` exige checkpoint treinado. Temperatura e top-k são constantes no runner, não flags de CLI.

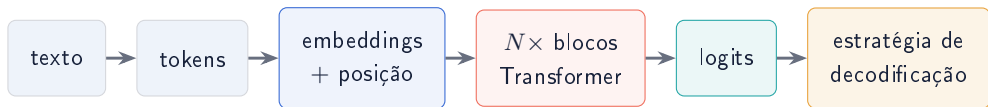
Experimento controlado

1. fixe checkpoint, prompt e seed;
2. varie temperatura: 0,4, 0,7, 1,2;
3. fixe 0,7 e varie top-k: 4, 12, 32;
4. compare repetição, diversidade e coerência;
5. repita com outras seeds antes de concluir.

Loss e perplexidade avaliam o modelo; as estratégias de amostragem alteram a continuação gerada sem mudar seus parâmetros. Uma única amostra é evidência fraca.

Fonte: `run_tiny_shakespeare.py`.

Reconstruindo o pipeline completo



1

Comunicação direta

attention combina posições permitidas com pesos aprendidos.

2

Contexto

self-attention torna cada vetor dependente da sequência.

3

Paralelismo

sem recorrência, posições treinam juntas por camada.

Pipeline mostrado para um decoder de linguagem: greedy ou amostragem transforma logits na escolha do próximo token.